

Lezione S4 — Binary Exploits

Lezione — Modulo S4: Binary Exploits e Buffer Overflow

Corso: Lab Sicurezza Informatica T **Materiale:** “Binary exploits” (13 marzo, 64 pp.) · “LAB bruteforcing e buffer overflows” (18 marzo, 55 pp.) **Prerequisiti:** S1 (enumerazione, GDB di base); architettura IA32 (registri, segmenti processo); compilazione gcc **Nota esame:** ★ **tipologia d’esame — Binary exploitation;** conta per il quiz teorico (40%); penalità per risposta sbagliata — non rispondere se non sei sicuro.

Come leggere questa lezione

Il filo conduttore è lo **stack frame di una funzione C in IA32**: capire cosa c’è sulla pila quando una funzione viene chiamata è la chiave per capire sia come funziona l’attacco sia come vengono costruite le difese. Tutto il resto — shellcode, NOP sled, return-to-libc, ROP — deriva da questo punto di partenza. I ganci concreti sono gdb, gcc e il Perl one-liner per generare payload.

Visione d’insieme — threat model

Prospettiva attaccante

Un programma C che accetta input esterno (da tastiera, da rete, da file) spesso copia quell’input in un buffer di dimensione fissa senza controllarne la lunghezza. Il linguaggio C non effettua questo controllo automaticamente: è responsabilità del programmatore. Quando manca, l’attaccante può fornire una stringa più lunga del buffer, e i

byte in eccesso sovrascrivono ciò che si trova nello stack immediatamente dopo: il valore salvato di EBP del chiamante, e poi — il punto critico — il **return address**, l'indirizzo a cui il processore salterà quando la funzione eseguirà la RET.

Sostituire il return address con un indirizzo a propria scelta significa dirottare il flusso di esecuzione verso codice arbitrario. A seconda delle protezioni attive, l'attaccante può: - puntare a **shellcode iniettato nel buffer stesso** (se lo stack è eseguibile); - puntare a una **funzione di libreria già in memoria** come `system()` — Return-to-Libc — anche se lo stack non è eseguibile; - concatenare **gadget ROP** dal segmento `.text` del binario, che non è soggetto ad ASLR nelle configurazioni di base.

L'obiettivo finale tipico è aprire una shell con i privilegi del processo vittima — che in scenari SUID significa diventare root.

Prospettiva difensore

Le contromisure operano a livelli diversi: il compilatore inserisce i **canarini** per rilevare la sovrascrittura prima del RET; l'OS marca lo stack come **non eseguibile** (NX) per bloccare lo shellcoding; l'OS randomizza il layout degli indirizzi (**ASLR**) per rendere imprevedibili stack base e indirizzi di libreria; l'eseguibile può essere compilato come **PIE** per randomizzare anche il segmento `.text`. Ogni protezione blocca solo alcune tecniche: ROP usa codice legittimo già caricato, aggirando NX; i gadget da `.text` restano prevedibili senza PIE; i canarini si aggirano con brute force su processi che forkano. La frontiera attuale è **CFI (Control Flow Integrity)**.

Lo stack frame e la convenzione `__cdecl`

Il punto di partenza è capire cosa accade in memoria quando una funzione viene chiamata. La convenzione di chiamata C (detta `__cdecl` in IA32) funziona così: il chiamante spinge i **parametri** sullo stack in ordine inverso, poi esegue `CALL` che salva automaticamente il **return address** (l'indirizzo dell'istruzione successiva nel chiamante).

All'ingresso, la funzione chiamata esegue il **function prologue**: `PUSH EBP` (salva il base pointer del chiamante), `MOV EBP, ESP` (imposta il proprio frame), `SUB ESP, N` (riserva spazio per le variabili locali, buffer inclusi).

Il risultato è che sullo stack, guardando dagli indirizzi bassi agli alti, si trovano in ordine: variabili locali e buffer → EBP salvato del chiamante (4 byte) → return address (4 byte) → parametri della funzione. La funzione al termine esegue `MOV ESP, EBP`, `POP EBP`, `RET` — quest'ultima estrae il return address dallo stack e lo carica in EIP.

Il punto critico dell'architettura è questa asimmetria: lo **stack cresce verso indirizzi decrescenti** (ogni `PUSH` decrementa ESP di 4), ma il **riempimento di un buffer avviene verso indirizzi crescenti** — è esattamente questa asimmetria che rende possibile il buffer overflow. Scrivendo oltre i limiti del buffer, i byte in eccesso sovrascrivono EBP salvato e poi il return address, che si trovano a indirizzi più alti.

gets e strcpy — la radice della vulnerabilità

Le funzioni `gets(buffer)` e `strcpy(dest, src)` sono le protagoniste del lab perché non effettuano alcun controllo sulla lunghezza dell'input. `gets` legge da `stdin` fino a `\n` o EOF e scrive nel buffer senza limiti. `strcpy` copia byte per byte fino al terminatore `\0`, anche se la destinazione non ha spazio sufficiente.

Quando scrivi 150 caratteri in `buf[100]`, i 50 byte in eccesso sovrascrivono ciò che viene dopo il buffer nel frame: prima EBP salvato (4 byte), poi il return address (4 byte). Questo è prevedibile e sfruttabile: se sai la struttura del frame, sai esattamente quanti byte di padding servono per raggiungere il return address e scrivere sopra un indirizzo controllato.

La vulnerabilità non è nella funzione in sé ma nel fatto che il linguaggio C lascia questa responsabilità al programmatore — e i programmatori dimenticano. Le alternative sicure (`fgets(buf, sizeof(buf), stdin)`, `strncpy(dest, src, n)`) impongono un limite esplicito.

⚠ **Pattern frequente da errori_frequenti.md:** costruisci sempre la catena “che informazione ho → cosa cerco → quale comando la trova”. Nel BOF: “ho un buffer di N byte → cerco quanti byte servono prima che il return address venga sovrascritto → uso GDB con payload di lunghezza crescente per trovarlo”.

`gdb` — il microscopio del reverse engineer

`gdb ./binario` è lo strumento con cui l'attaccante capisce cosa fa il programma e dove si trovano le strutture dati in memoria. I comandi essenziali del modulo S4 sono:

`disas main` e `disas vuln` mostrano il disassembly delle funzioni: si legge l'assembly generato dal compilatore, si identificano le chiamate alle funzioni pericolose, le dimensioni dei buffer (visibili in `SUB ESP, N`), e gli indirizzi delle funzioni “nascoste” che non vengono mai chiamate dal flusso lecito.

`run $(perl -e 'print "A"x120')` esegue il programma passando 120 caratteri 'A'. Se crasha con SIGSEGV e GDB riporta `0x41414141 in ?? ()`, i 120 caratteri erano abbastanza da sovrascrivere il return address con quattro 'A' (0x41 è ASCII di 'A'). Se invece esce normalmente, il buffer non era ancora pieno. Con bisezione — 100, 120, 110, 112 — si trova il punto esatto. Quando invece di `0x41414141` il GDB mostra `0x42424242`, significa che le 'B' di controllo hanno sovrascritto esattamente il return address: l'offset è trovato.

`x/200xw $esp` fa un dump della memoria a partire da ESP, mostrando 200 word in formato esadecimale. In questo dump vedi le tue 'A' (`0x41414141`), le 'B' che marcano il return address (`0x42424242`), e poi — dopo aver iniettato il payload reale — le NOP (`0x90909090`) e lo shellcode. Serve per scegliere un indirizzo di ritorno plausibile che cada nella slitta di NOP.

`info functions` elenca le funzioni del binario con i loro indirizzi. Prima del `run` mostra i simboli noti; dopo il `run` include anche le librerie caricate. La tattica raccomandata: annotare i nomi interessanti prima, poi cercare con `info functions secret` (o regex simile) dopo il `run` per avere l'indirizzo reale.

`b *main` imposta un breakpoint su main; `p system`, `p exit` stampano gli indirizzi delle funzioni di libreria; `x/500s $esp` cerca stringhe nello stack — utile per trovare la variabile d'ambiente `SHELL=/bin/sh`.

Little endian — perché l'indirizzo va scritto “al contrario”

L'architettura IA32 (Intel) usa la convenzione **little endian**: il byte meno significativo di un dato multi-byte viene memorizzato all'indirizzo più basso. Quando costruisci un payload e vuoi sovrascrivere il return address con l'indirizzo `0x0804D432`, nella stringa di attacco devi fornire i byte in ordine inverso: `\x32\xD4\x04\x08`.

L'esempio concreto del lab: l'indirizzo della funzione `valid_code` è `0x0804D432`, quindi la stringa d'attacco è `"AAAAAAAA BBBB \x32\xD4\x04\x08"`. Lo stesso vale per ogni indirizzo che inserisci nel payload: funzioni del binario, indirizzi di `system()`, start di shellcode, gadget ROP. Sbagliare l'endianness è uno degli errori più comuni nei BOF.

Shellcode e NOP Sled — iniettare ed eseguire codice

Il **shellcode** è una sequenza di byte che rappresenta istruzioni assembly, progettate per essere eseguite direttamente dal processore. In IA32 su Linux, il modo canonico per aprire una shell usa la syscall `execve("/bin/sh")` tramite l'interruzione software `int 0x80`: si carica l'identificatore della syscall in EAX (11 per `execve`), i parametri in EBX/ECX/EDX, e si esegue `int 0x80`. Il kernel intercetta e chiama la syscall.

Il vincolo pratico fondamentale è che lo shellcode non può contenere **byte nulli** (`\x00`): `gets` e `strcpy` usano il terminatore di stringa `\0` per fermarsi, quindi un byte nullo tronca il payload prima che arrivi a destinazione. Questa è l'operazione di **Zeros Cut-off**: le istruzioni che producono byte vanno sostituite con equivalenti che non li producono. Per esempio, `mov EBX, 0` si codifica con byte nulli, ma `xor EBX, EBX` (XOR di un registro con se stesso produce 0) non ne produce. Allo stesso modo, `mov EAX, 1` genera byte nulli, ma `xor EAX, EAX; mov AL, 1` (AL è il byte basso di EAX) no.

Lo shellcode usato nel lab è lungo **46 byte** e lancia `/bin/sh`. La dimensione si verifica con Python: `python → >> len(b'..SHELLCODE..')`.

Il problema del salto preciso si risolve con il **NOP Sled**: si riempie il buffer con decine di istruzioni NOP (`\x90`) prima dello shellcode. La CPU esegue i NOP senza fare nulla e avanza di un byte alla volta fino al

primo byte di shellcode. Quindi invece di puntare esattamente all'inizio dello shellcode — che varia leggermente tra esecuzioni — è sufficiente “atterrare” in qualsiasi punto della slitta.

La struttura del payload nell'esercizio 3 del lab è: **66 NOP + 46 byte shellcode + 4 byte return address = 116 byte totali**. L'indirizzo di ritorno si sceglie leggendo il dump di `x/200xw $esp` e identificando la zona dove partono le NOP (`0x90909090`): un qualsiasi indirizzo in quella zona va bene.

L'ambiente richiede il bit **SUID** sull'eseguibile per dimostrare l'escalation a root: `chown root:root es; chmod u+s es`. Così, quando lo shellcode apre `/bin/sh`, la shell eredita l'UID effettivo di root — visibile con `id` che mostrerà `uid=0(root)`.

Return-to-Libc — quando lo stack non è eseguibile

Se il binario viene compilato senza `-z execstack`, lo stack è non eseguibile (flag NX attivo). Tentare di eseguire shellcode iniettato causa immediatamente un'eccezione di protezione della memoria.

La risposta è **Return-to-Libc (ret2libc)**: invece di eseguire codice iniettato, si usa la sovrascrittura del return address per saltare a codice **già caricato in memoria** — specificamente a `system()` della libreria C standard (libc), sempre presente in qualsiasi processo Linux. L'attaccante non inietta istruzioni eseguibili: inietta solo **dati** (indirizzi e parametri) che dirigono l'esecuzione verso funzioni legittime.

Per costruire il payload `ret2libc` servono tre elementi, trovabili da GDB con breakpoint su `main`: `- p system` → indirizzo di `system()` - `p exit` → indirizzo di `exit()` (garantisce terminazione pulita, comportamento meno anomalo) - `x/500s $esp` → scansiona lo stack per trovare la stringa `"/bin/sh"` nella variabile d'ambiente `SHELL`

Il payload ha struttura: **[padding di N byte] + [indirizzo system] + [indirizzo exit] + [indirizzo stringa `"/bin/sh"`]**. Quando RET salta a `system()`, la convenzione `__cdecl` prevede che sullo stack ci sia già l'indirizzo di ritorno (qui: `exit`) e poi l'argomento (`"/bin/sh"`).

⚠ Due trabocchetti pratici: (1) la variabile d'ambiente è `SHELL=/bin/sh`, non solo `/bin/sh` — l'indirizzo della stringa punta all'inizio di `SHELL=`, quindi va aggiunto un offset di 6 byte per puntare al `/` iniziale; (2) se

l'indirizzo di `system()` contiene un byte `\x00`, la stringa viene troncata — il workaround è usare un indirizzo di 1-2 byte più avanti nell'istruzione (l'effetto è identico).

La ricompilazione senza stack eseguibile: `gcc -o es_nostack -fno-stack-protector -m32 es.c` (senza `-z execstack`).

ROP — Return-Oriented Programming

Quando ASLR randomizza gli indirizzi di libreria e `ret2libc` diventa difficile perché gli indirizzi cambiano a ogni esecuzione, entra in gioco **Return-Oriented Programming (ROP)**: costruisce il codice da eseguire assemblando “gadget” già presenti nel segmento `.text` del binario.

Un **gadget ROP** è qualsiasi sequenza di istruzioni del binario che termina con `RET`. Il segmento `.text` non è soggetto ad ASLR senza PIE — gli indirizzi dei gadget sono prevedibili. L'attaccante cerca gadget che eseguano operazioni utili: `POP EBX; RET`, `MOV EAX, EBX; RET`, `INT 0x80`, ecc. La catena di gadget viene costruita sullo stack.

Il meccanismo sfrutta `RET` stessa: `RET` carica EIP dal valore in cima allo stack e incrementa ESP. Se sullo stack si trova la sequenza `[addr_gadget1 | dato1 | addr_gadget2 | dato2 | ...]`, ogni gadget esegue la sua operazione atomica e poi `RET` salta al gadget successivo. Si ottiene così un “programma” costruito interamente da frammenti di codice legittimo — bypassando NX, perché si eseguono istruzioni già nel segmento `.text`, non sullo stack.

RET2SYSCALL è la versione più diretta: trovare nel binario gadget che carichino EAX con il numero di syscall, EBX/ECX/EDX con i parametri, e poi un gadget con `INT 0x80`. Poiché ogni funzione che usa certi registri li ripristina con una sequenza di `POP`, gadget utili come `POP EBX; RET` e `POP EAX; RET` si trovano ovunque.

Contromisure — meccanismo e limiti

Canarini (stack canary): un valore casuale di 4 byte inserito tra le variabili locali e l'EBP salvato ad ogni chiamata di funzione; il `RET` verifica l'integrità prima di procedere. Un overflow che raggiunge il return address deve necessariamente sovrascrivere anche il canarino — non può “scavalcarlo”. Se il canarino è cambiato, il processo viene

terminato. Limit: su processi che forkano, il canarino viene copiato identico nel figlio; con brute force si indovina un byte alla volta senza che il padre se ne accorga. Si disabilita con `-fno-stack-protector`.

NX Stack (No-eXecute / XD bit): il processore (su architetture a 64 bit) o l'OS marca le pagine di stack come non eseguibili. Tentare di eseguire codice sullo stack causa un'eccezione immediata. Importante: **non disponibile sui processori Intel/AMD a 32 bit nativamente**; disponibile su x86_64 dal kernel 2.6.8. Non blocca `ret2libc` né ROP, che usano codice legittimo. Si forza lo stack eseguibile con `-z execstack`.

ASLR (Address Space Layout Randomization): randomizza a ogni avvio gli indirizzi base di librerie, stack e heap. **Non randomizza .text** nelle configurazioni di base — questo è il motivo per cui ROP (che usa gadget da `.text`) funziona anche con ASLR. Si disabilita scrivendo `echo 0 > /proc/sys/kernel/randomize_va_space` (richiede root).

PIE (Position Independent Executable): estende la randomizzazione anche a `.text`. Richiede compilazione come `position-independent code`. L'indirizzamento delle funzioni usa le tabelle PLT e GOT — che però sono in memoria scrivibile e in alcuni scenari sovrascrivibili. Il hardening aggiuntivo è **RelRO** (read-only GOT).

CFI (Control Flow Integrity): famiglia di tecniche che impone vincoli su dove EIP può puntare durante l'esecuzione. L'implementazione hardware più moderna usa **puntatori cifrati**: il processo sceglie una chiave all'avvio e la CPU cifra/decifra trasparentemente tutti gli indirizzi di ingresso/uscita da funzioni. Un programma vulnerabile può essere fatto crashare (DoS) ma non dirottato. Esempio reale: Pointer Authentication Codes (PAC) sui processori Apple $\geq$ A12.

Connessioni

- **Con SysAdmin**: la configurazione errata più comune che abilita gli exploit locali è il bit **SUID** su binari con codice vulnerabile. Un SUID binary eseguito da utente non privilegiato esegue come root — se ha un BOF, la shell iniettata eredita i privilegi root. In 3A/systemd si gestiscono servizi che girano come root e hanno questa superficie d'attacco.
- **Con S1 (Enumerazione)**: in S1 si scansionavano servizi aperti e si raccoglievano banner; in S4 si sfruttano quei servizi con BOF remoto (esempio: `secret_function_remote` con `nc -l -p 8000 -e ./es`). La catena `nmap` → `banner` → `exploit` è la catena offensiva reale.

- **Con S7 (Backdoor) e S11 (privesc):** il BOF è uno dei vettori classici di privilege escalation. S7 e S11 usano tecniche simili in scenari più articolati (NIDS, misconfiguration, persistence).
-

Domande di autoverifica (stile quiz teorico)

Penalità per risposta sbagliata — se non sei sicuro, non rispondere.

1. [Vero/Falso] Il return address di una funzione si trova sullo stack a un indirizzo **più alto** rispetto alle variabili locali della stessa funzione in IA32.

Vero. Lo stack cresce verso il basso (indirizzi decrescenti): le variabili locali vengono allocate dopo l'ingresso nella funzione (a indirizzi bassi), mentre il return address è stato pushato prima dell'ingresso (a indirizzo più alto). Il buffer overflow scrive verso indirizzi crescenti, raggiungendo prima EBP salvato e poi il return address.

2. [Multipla] In un attacco ret2libc su IA32 con `__cdecl`, qual è la struttura corretta del payload dopo il buffer di padding?

1. [addr system] + [argomento] + [addr exit]
2. [addr system] + [addr exit] + [argomento "/bin/sh"]
3. [argomento] + [addr exit] + [addr system]
4. [addr exit] + [addr system] + [argomento]

Risposta: b. Quando RET salta a `system()`, la convenzione `__cdecl` impone che lo stack contenga return address (qui exit) poi l'argomento. Quindi: padding + addr_system + addr_exit + addr_stringa.

3. [Vero/Falso] ASLR, in configurazioni di base senza PIE, randomizza gli indirizzi del segmento `.text` del binario ad ogni esecuzione.

Falso. ASLR randomizza indirizzi base di librerie, stack e heap — ma **non** `.text`. Gli indirizzi delle funzioni del binario restano fissi tra le esecuzioni. Questo è ciò che rende praticabili gli attacchi ROP senza PIE.

4. [Multipla] Il NOP Sled ($\backslash x90 \times N$) viene inserito nel payload per:

1. aumentare la dimensione del payload fino alla soglia di overflow

2. rendere meno necessario conoscere l'indirizzo esatto di inizio dello shellcode
3. evitare byte nulli nel payload
4. aggirare la protezione NX dello stack

Risposta: b. La CPU esegue i NOP senza effetti e avanza EIP di un byte. Se il return address punta a qualsiasi punto della slitta invece che all'esatto inizio dello shellcode, la CPU scivola lungo i NOP fino a raggiungerlo.

5. [Vero/Falso] Un canarino di stack può essere aggirato su un processo che forka figli perché ogni fork eredita una copia identica del canarino del processo padre.

Vero. Il canarino viene generato all'avvio del processo e copiato nella fork. L'attaccante può brute-forcarlo un byte alla volta: ogni tentativo sbagliato crasha solo il figlio, ma il padre genera un nuovo figlio con lo stesso canarino identico. Questo rende il brute force praticabile in scenari server che restano in ascolto.

Riepilogo

Il return address è la chiave del controllo del flusso. In IA32 con `__cdecl`, il return address è sullo stack ed è scrivibile: un buffer overflow che lo raggiunge dà controllo su EIP. Tutto il resto è un modo di sfruttare questo controllo — shellcode se lo stack è eseguibile, ret2libc se non lo è, ROP per aggirare anche ASLR.

Le contromisure si combinano e si aggirano in sequenza. Canarini bloccano l'overflow più banale; NX blocca lo shellcoding; ASLR rende imprevedibili gli indirizzi di libreria; PIE randomizza anche il codice. Ma ogni protezione ha un limite: ROP bypassa NX; `.text` fisso bypassa ASLR; fork + brute force aggira i canarini. La risposta attuale è CFI.

L'ambiente di laboratorio è artificialmente permissivo. I flag `-fno-stack-protector`, `-z execstack`, `echo 0 > /proc/sys/kernel/randomize_va_space` e il bit SUID vengono impostati manualmente per permettere gli esercizi. Su sistemi moderni tutte queste protezioni sono attive di default — capire perché esistono e come interagiscono è ciò che l'esame valuta.



Hub: [[master_map_studio]] · [[concept_maps]] ·
[[metodo_studio_esami_pratici]]